



NAZARICK OVERSEER

Dossier de Projet • Titre Professionnel DWWM • Cys Enzo

Ce document présente la synthèse complète du projet **Nazarick Overseer**, un jeu web persistant de type "idle/incremental" s'inspirant de l'univers d'*Overlord*. Conçu de bout en bout pour valider l'ensemble des compétences (CP1 à CP8) du titre de Développeur Web et Web Mobile (DWWM), il s'agit d'une application fonctionnelle garantissant une boucle de jeu addictive, sécurisée et pérenne.

1. CONCEPT ET IDENTITÉ

Le joueur est placé dans le rôle d'un « Overseer » chargé de développer la Grande Tombe de Nazarick. L'économie du jeu repose sur une dynamique de croissance continue, y compris hors ligne, structurée autour de ressources et d'entités spécifiques :

- **Génération de 3 ressources clés** : l'Or (cercle doré), le Mana (losange bleu) et les Matériaux (triangle blanc).
- **Contenu évolutif** : Recrutement de ~22 Gardiens qui sont responsables de générer continuellement les ressources (Or, Mana, Matériaux). Ces Gardiens sont répartis sur 3 branches, avec un déblocage successif via l'acquisition de 5 Intendants. L'optimisation est assurée par 8 améliorations stratégiques agissant comme des multiplicateurs.
- **Progression hors-ligne** : L'algorithme calcule la production générée en l'absence du joueur au moment de sa reconnexion, de manière sécurisée et plafonnée à 24 heures pour limiter les abus temporels.
- **Classement compétitif** : Un système dynamique permet aux joueurs de se situer par rapport aux autres Overseers en se basant sur le volume total de leurs ressources.

2. ARCHITECTURE TECHNIQUE (STACK)

Le choix délibéré a été fait de ne pas utiliser de framework lourd (tel que Symfony ou Laravel), privilégiant une approche native pour démontrer une maîtrise algorithmique et structurelle absolue.

Back-End & Logique

PHP 8 Natif

POO

MVC

- **Séparation des responsabilités** : Architecture MVC stricte (Contrôleurs pour l'orchestration, Services pour les règles métier, Repositories pour les requêtes).
- **Routeur personnalisé** : Implémentation d'un routeur gérant point d'entrée unique et dispatching.

Front-End & Expérience

JavaScript Vanilla

AJAX

CSS3

- **Interactivité asynchrone** : Actualisation des données (tick de production) et actions de jeu via appels API (Fetch) sans aucun rechargement de page.
- **Dégradation gracieuse** : L'application demeure jouable avec JS désactivé via un repli sur des formulaires POST classiques.

3. BASE DE DONNÉES ET PERSISTANCE

Le système repose sur un moteur **MySQL / MariaDB** avec une modélisation poussée (MCD/MLD/MPD) assurée par PDO.

- **Tables associatives et intégrité** : Les relations complexes (joueur ↔ gardiens, joueur ↔ améliorations) utilisent des clés primaires composites (ex: id_utilisateur, id_gardien), garantissant nativement l'unicité et facilitant la gestion des niveaux.
- **Stockage optimisé** : Les ressources globales (Or, Mana, Matériaux) sont gérées directement sur l'entité Utilisateurs pour réduire les jointures coûteuses et maximiser les performances lors du recalcul hors-ligne.

4. SÉCURISATION APPLICATIVE (AUTORITÉ SERVEUR)

CONFORMITÉ ET PROTECTIONS SERVEUR

L'intégralité du calcul économique s'effectue sur le serveur ; le client Web n'effectue que de l'interpolation visuelle. L'application est protégée contre un large spectre de vulnérabilités :

- **Faillle Temporelle** : Utilisation de DateTimeImmutable et bornage strict par le serveur (plafond 24h).
- **Attaques CSRF** : Sécurité à double canal implémentée via un jeton unique en session validé par hash_equals() (contre les *Timing Attacks*). Le token est transmis via champ caché (formulaires) ou en-tête X-CSRF-Token (AJAX).
- **Injections (SQL & XSS)** : Utilisation stricte de requêtes préparées via PDO, et échappement systématique des entrées/sorties via htmlspecialchars.
- **Sécurisation de l'hébergement (Plesk)** : Pointer web ciblant uniquement public/, retrait de l'affichage des erreurs en production, masquage des informations système.

5. GESTION, ÉQUILIBRAGE ET ADMINISTRATION

Afin d'assurer une évolutivité de contenu sans altérer le code source, Nazarick Overseer inclut un **Back-Office (CRUD)** accessible exclusivement aux administrateurs. Il permet d'éditer, d'ajouter ou de supprimer des Gardiens, des Intendants et des Améliorations de façon dynamique.

Le jeu garantit par ailleurs un **équilibre économique continu**, basé sur une croissance de coût exponentielle ($\times 1.14$ par niveau) avec des limites logiques (niveau maximal plafonné à 50) traitées dans un boucle adaptative par le service métier.

6. BILAN ET PERSPECTIVES D'AMÉLIORATION

Ce projet démontre une compréhension globale et transversale du métier de développeur web, validant avec succès les exigences du Titre DWWM. Pensé de façon itérative, le code prévoit déjà des axes d'amélioration identifiés :

- **Assurance Qualité** : Intégration à venir de tests unitaires automatisés (via *PHPUnit*) pour la logique métier critique (algorithmique de coûts, anti-triche).
- **Persistance Polyglotte** : Conception finalisée d'une surcouche NoSQL (*MongoDB*) afin de gérer un journal persistant d'événements aléatoires (raids), séparant ainsi la donnée métier (SQL) de la donnée d'historique (NoSQL).
- **Accessibilité** : Poursuite de l'audit et de l'amélioration des contrastes (WCAG) sur l'interface sombre.